

Mysten PAS

Security Assessment

April 13th, 2026 — Prepared by OtterSec

Michał Bochnak	embe221ed@osec.io
Sangsoo Kang	sangsoo@osec.io
Thiago Tavares	thitav@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

General Findings **5**

OS-MPS-SUG-00 | PTB Template Stores Only One Command 6

OS-MPS-SUG-01 | Redundant Local Version State Synchronization 7

Appendices

Vulnerability Rating Scale **8**

Procedure **9**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Mysten Labs engaged OtterSec to assess the `pas` program. This assessment was conducted between March 10th and March 27th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 2 findings throughout this audit engagement.

In particular, we suggested utilizing a Command vector since a PTB consists of multiple commands chained together (OS-MPS-SUG-00). We further recommended removing local versioning and validating version state directly against `Namespace` to avoid redundant local version state synchronization (OS-MPS-SUG-01).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/MystenLabs/pas. This audit was performed against de0b95c.

A brief description of the program is as follows:

Name	Description
pas	A Sui framework for holding assets in Permissioned Asset Standard-controlled accounts and routing sends, withdrawals, and clawbacks through policy-defined approval workflows. It gives issuers deterministic accounts/policies, configurable compliance gates, optional clawback control, and a closed-loop transfer system for managed assets.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 2 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	0
INFO	2

© 2026 Otter Audits LLC. All Rights Reserved.

04 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MPS-SUG-00	The existing implementation only stores a single `ptb::Command` per action, even though resolving a PAS flow may require multiple chained PTB steps.
OS-MPS-SUG-01	Version information is duplicated between `Namespace` and local objects, which increases synchronization overhead and may briefly leave components out of sync after upgrades due to synchronization delays.

© 2026 Otter Audits LLC. All Rights Reserved.

PTB Template Stores Only One Command OS-MPS-SUG-00

Description

The existing implementation in `template::set_template_command` utilizes `ptb::Command`, which supports only a single PTB instruction, whereas PAS resolution may require multiple ordered steps within a single programmable transaction. Since a PTB consists of multiple chained commands, it is more appropriate to utilize `vector<Command>`.

>_ *sources/templates.move* RUST

```rust
/// Sets the PTB template for a given Action.
public fun set_template_command<A: drop>(
    templates: &mut Templates,
    _: internal::Permit<A>,
    command: Command,
) {
    let key = type_name::with_defining_ids<A>();
    if (dynamic_field::exists_(&templates.id, key)) {
        let _ = dynamic_field::remove<_, Command>(&mut templates.id, key);
    };

    dynamic_field::add(&mut templates.id, key, command);
}
```

Remediation

Utilize `ptb::Transaction` instead of `ptb::Command`.

Patch

Mysten acknowledged this issue.

© 2026 Otter Audits LLC. All Rights Reserved.

Redundant Local Version State Synchronization OS-MPS-SUG-01

Description

The current implementation keeps version state in both `Namespace` and per-object local copies, such as `Account` and `Policy`, creating redundant state that must be explicitly synchronized after each upgrade. This duplication increases write costs, introduces delays before consistency is achieved, and adds unnecessary operational complexity, rendering upgrade flows harder to maintain.

Remediation

Remove local versioning and validate version state directly against `Namespace`.

Patch

Mysten acknowledged this issue.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.